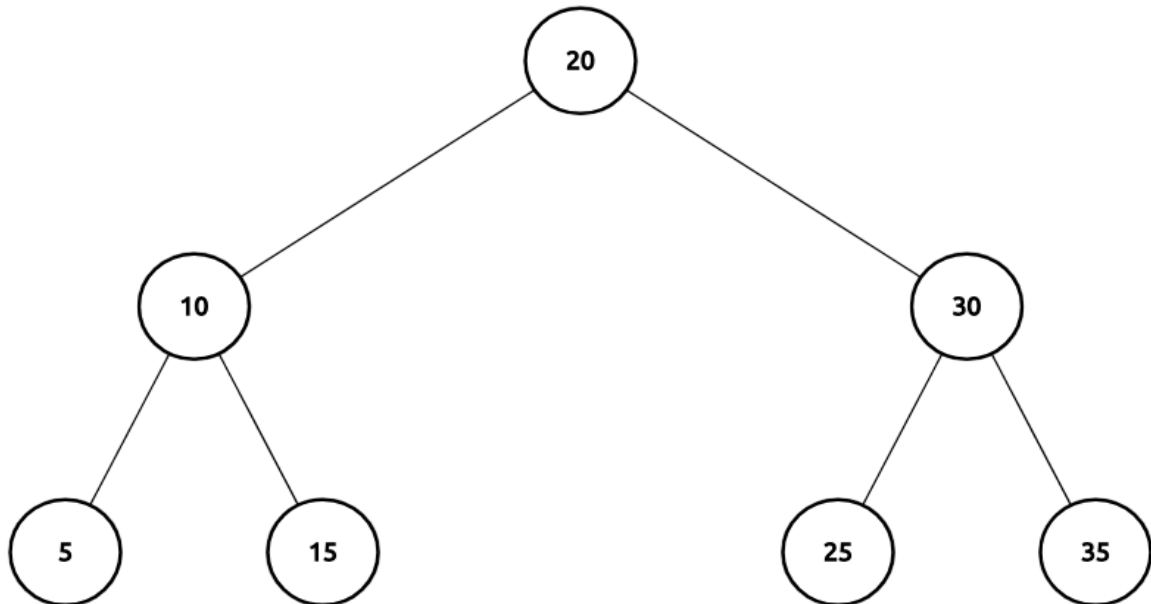


SimulEDA P1

1) Dada a AVL a seguir:

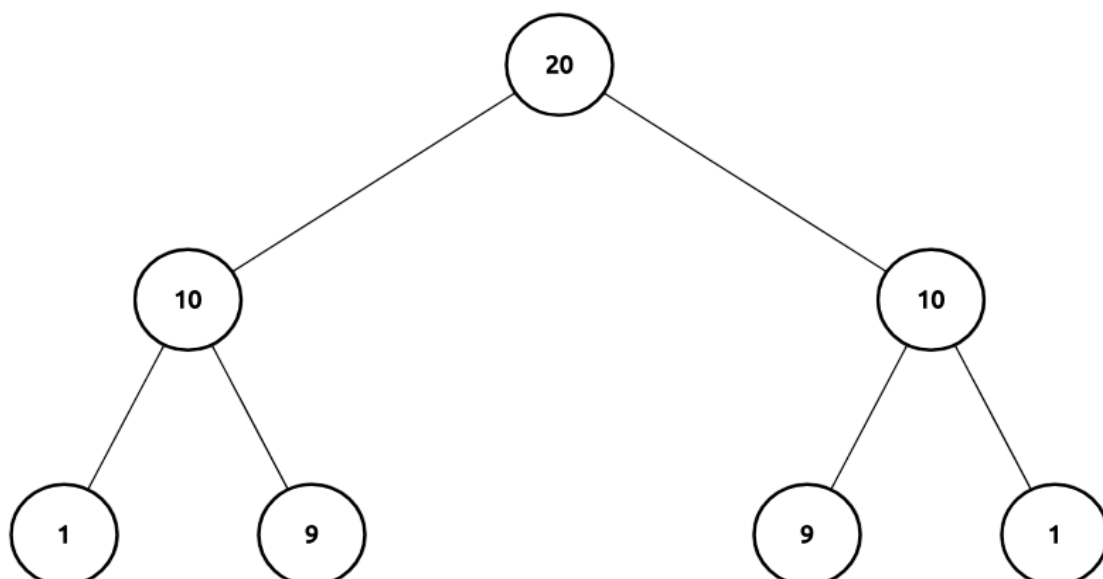


Indique a inserção incremental dos elementos: 27, 29, 26, 28, 12, 14, 40, 37, 1, 4, 11, 13, 9, 3

Indique a retirada incremental dos elementos: 20, 5, 9, 40, 29, 28

2) Faça uma função que verifique se uma árvore binária é simétrica.

Exemplo:



```
int eh_simetrica(TAB *a);
```

- 3) Faça uma função que verifique se todas as folhas de uma árvore binária estão no mesmo nível.

```
int mesmo_nivel(TAB *a);
```

- 4) Faça uma função que verifique se uma árvore binária de busca segue o padrão AVL.

```
int eh_avl(TAB *a);
```

- 5) Faça uma função que retorne todos os caminhos que somam n em uma árvore binária de busca. (Use a biblioteca de lista encadeada e retorne um vetor com todas as listas de caminhos)

```
TLSE** caminhos(TAB *a, int n);
```

- 6) Faça uma função que encontre um caminho entre dois vértices de um grafo.

```
TLSE* caminho(TG *g, int origem, int destino);
```

- 7) Faça uma função que encontre **TODOS** os caminhos entre dois vértices de um grafo.

```
TLSE** caminhos(TG *g, int origem, int destino);
```

- 8) Faça uma função que encontre o menor caminho entre dois vértices de um grafo.

```
TLSE* menor_caminho(TG *g, int origem, int destino);
```

- 9) Suponha que a estrutura de árvore binária (TAB) possui um campo vizinho que aponta para o nó ao lado.

```
typedef struct arv {  
    int info;  
    struct arv *esq, *dir, *prox;  
} TAB;
```

Crie uma função que ao receber uma árvore binária, faça as mudanças necessárias para retorná-la "encadeada".

Exemplo:

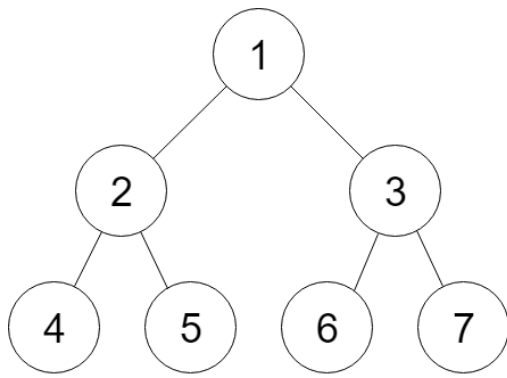


Figure A

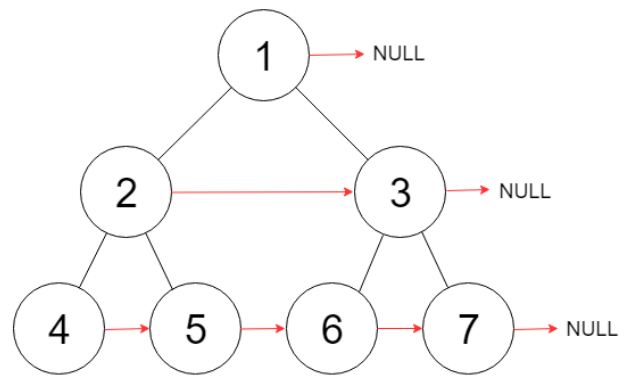


Figure B

TAB* encadeador(TAB* a);

10) Gere as partições inserindo os seguintes elementos:

31	12	84	45	85	23	97	86
30	54	25	56	79	66	59	75
26	48	68	2	24	72	5	7

- Siga o algoritmo de seleção com substituição. Assuma que a memória suporta 5 registros.
- Agora, refaça usando o algoritmo de seleção natural, com reservatório de tamanho 3 e memória que suporte 5 registros.